

Un ordinateur est une machine électronique programmable servant au traitement de l'information codée sous forme binaire, c'est-à-dire sous forme de tout ou rien (soit le courant passe, soit il ne passe pas).

Comme l'ordinateur a l'avantage d'exécuter très rapidement et sans erreurs les ordres qu'on lui donne (les instructions), il exécute beaucoup de traitements complexes plus vite et plus sûrement qu'un homme. Pour donner des ordres à l'ordinateur, il est nécessaire de pouvoir communiquer avec lui. Cette communication passe par un langage de programmation, dans lequel est écrit le programme.

1. Définition d'un Programme

Un programme est un assemblage et un enchaînement d'instructions élémentaires écrit dans un langage de programmation, et exécuté par un ordinateur afin de traiter les données d'un problème et renvoyer un ou plusieurs résultats.

2. Langages de Programmation

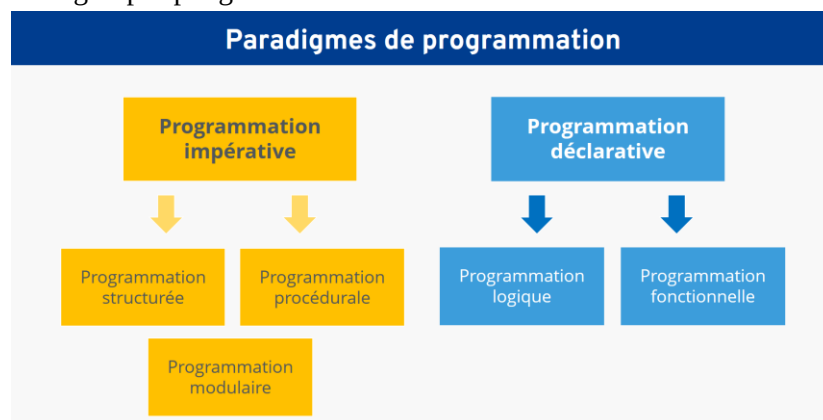
Ensembles de mots clefs et de règles pour former des phrases pouvant être traduites par la machine (binaire). On peut classer les langages de programmation selon :

- ✓ Langages de bas niveau : instructions élémentaires très proches de la machine (ex : Assembleur : V AX, [0110] signifie copier le contenu de 0110 dans le registre AX ; langage machine)
- ✓ Langages de haut niveau : instructions plus abstraites (C, C++, Perl, Java, Python).

3. Paradigmes de Programmation

Les paradigmes en programmation désignent les principes fondamentaux du développement de logiciels. Au mieux, on peut se les représenter comme des **styles de programmation fondamentaux divers** générant des codes logiciels de structure différente.

Ainsi, un paradigme de programmation est un style de programmation informatique qui traite de la manière dont les solutions aux problèmes doivent être formulées dans un langage de programmation. Les paradigmes sont regroupés par grandes familles :



Vue d'ensemble des principaux paradigmes de Programmation

3.1. Programmation Impérative

La programmation impérative est considérée comme « traditionnelle ». Les premiers langages de programmation et, de ce fait, les premiers programmes informatiques, ont été intégralement conçus sur cette base. Les *programmations structurée, procédurale et modulaire* sont trois approches majeures d'écriture et de structuration du code logiciel selon le paradigme programmation impérative. Exemple : Pascal, C, C++, Java, Python, PHP...etc.

3.2. Programmation Déclarative

L'approche déclarative a permis l'élaboration d'un paradigme alternatif à la programmation de code. Le principe de base de la programmation déclarative réside ici dans la *description du résultat final souhaité*. Il s'agit donc en premier lieu de l'objectif à atteindre, et non du *déroulement des étapes de résolution* comme c'est le cas en programmation impérative. En programmation déclarative, le code est plus difficile à comprendre en raison de son *haut niveau d'abstraction*, mais est également plus court et plus précis. Il existe des différences plus importantes entre les sous-catégories du paradigme de programmation déclarative qu'entre celles du style impératif. En outre, leur définition ou répartition n'est pas toujours très précise. Les approches principales du paradigme programmation déclarative sont la *programmation fonctionnelle* (Python, Lisp,...), la *programmation logique* (Prolog, Python, ...) et la *programmation descriptive* (HTML, XML, LaTeX).

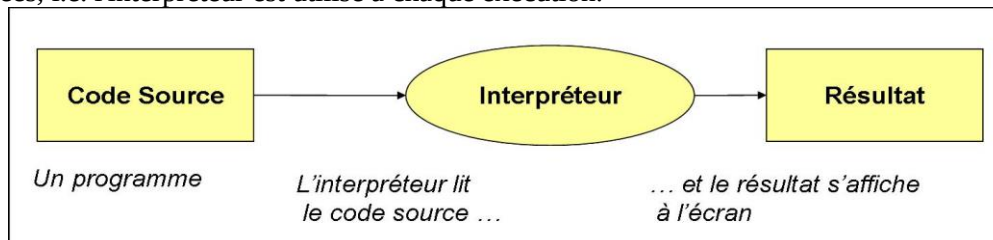
3.3. Programmation Orientée objet

Programmation consistant en la définition et l'assemblage de briques logicielles appelées objets : C++, Java, Python, PHP, SMALLTALK, O-CAML...

4. Production de Programmes

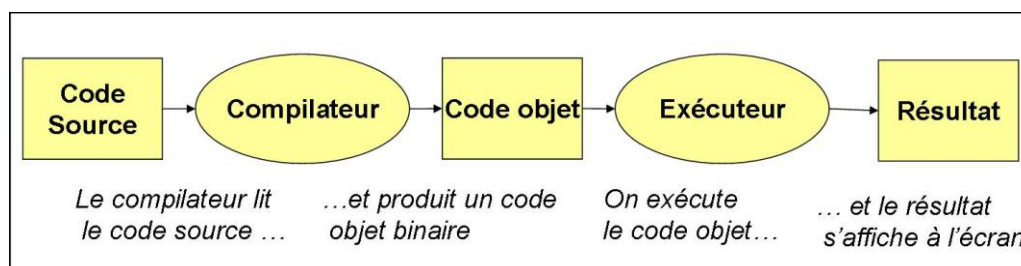
4.1. Interprétation

Chaque ligne du code source est traduite au fur et à mesure en instructions qui sont directement exécutées, i.e. l'interpréteur est utilisé à chaque exécution.



4.2. Compilation

Action de transformer un code source en code binaire, de manière à obtenir un fichier exécutable à partir d'un fichier source. Compilation = traduction du code écrit dans un langage dit source en langage objet ou cible (analyse lexicale, syntaxique, sémantique et production du code objet). L'Édition de liens ...



5. Définition d'un Algorithme

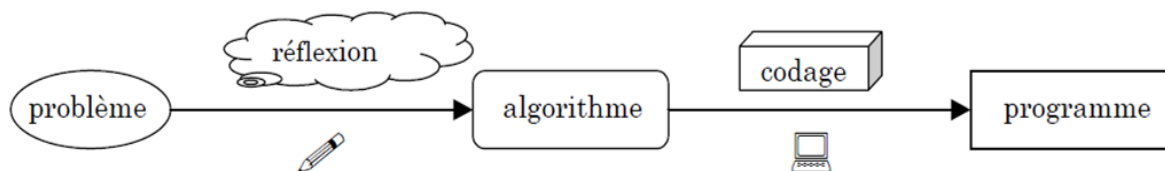
Le mot algorithme provient du nom du célèbre mathématicien arabe : Muhammad ibn Musa Al Kwarizmi (780-850). Son nom donna au moyen-âge le nom "algorisme" qui devint algorithme avec Lady Ada Lovelace (1792-1871).

Un algorithme est une suite finie d'opérations élémentaires, à appliquer dans un ordre déterminé, à des données. Sa réalisation permet de résoudre un problème donné. Ainsi, il représente l'enchaînement des actions (instructions) nécessaires pour faire exécuter une tâche à un ordinateur (résoudre un problème). Un algorithme s'écrit le plus souvent en *pseudo-langage* de programmation (*appelé langage algorithmique*). Un algorithme n'est donc exécutable directement par aucune machine. Mais il a l'avantage d'être traduit facilement dans tous les langages de programmation.

6. Définition d'Algorithmique

L'algorithmique, l'art d'écrire des algorithmes, permet de se focaliser sur la procédure de résolution du problème sans avoir à se soucier des spécificités d'un langage particulier. L'algorithmique est une méthode de réflexion qui permet de décomposer un problème complexe en une suite ordonnée et finie d'actions à accomplir (l'algorithme), pour parvenir à un résultat déterminé.

Pour résoudre un problème, il est vivement conseillé de réfléchir d'abord à l'algorithme avant de programmer proprement dit, c'est à dire d'écrire le programme en langage de programmation.



7. La Démarche Algorithmique

L'algorithmique est une méthode permettant de trouver l'algorithme qui résout un problème donné. L'algorithme lui-même n'est donc que le résultat d'une démarche qui consiste à :

- **Comprendre le Problème à Résoudre**

Si ce n'est pas le cas, il pourra être utile de se le représenter à travers des cas ou exemples concrets. Par ailleurs, le questionnement et la recherche d'une reformulation seront aussi précieux pour éclairer le sujet.

- **Rechercher une Solution**

Il n'en existe pas forcément qu'une, mais certaines sont meilleures que d'autres. Selon la formule consacrée, un moyen efficace de résolution sera de « Diviser chaque problème en autant de parties qu'il est nécessaire pour mieux les résoudre ».

8. Propriétés d'un Algorithme

- Un algorithme doit :
 - ✓ Se terminer après un nombre fini d'opérations,
 - ✓ Fournir un résultat.
- Chaque opération doit être :
 - ✓ Définie rigoureusement et sans ambiguïté
 - ✓ Effective, c.-à-d. réalisable par une machine
- Un même problème peut être résolu au moyen de plusieurs algorithmes.

9. Représentation d'un Algorithme

Un algorithme est généralement exprimé dans un langage *informel*, ou incomplètement formalisé : texte libre (i.e. description des différentes étapes en langue naturel : français, arabe...etc.), **organigramme** (diagramme représentant les étapes), **pseudo-code** (version simplifiée d'un langage informatique) ou

autres. Par opposition, une démonstration mathématique ou un programme informatique sont exprimés en utilisant des *langages formels*. Cela signifie donc que l'écriture d'un algorithme est souple, elle vise à exprimer une méthode de résolution de façon compréhensible à un être humain. Mais pour la même raison, un algorithme ne peut pas être traité directement par un ordinateur : il doit être formalisé, i.e. transformé en un programme.

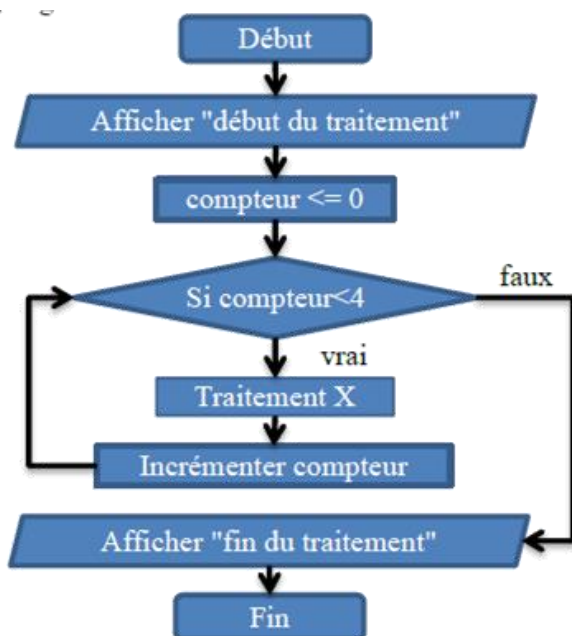
✓ **L'Organigramme** : Il n'existe pas vraiment de norme pour les organigrammes représentant des algorithmes. On peut tout de même mentionner certains points qui font consensus :

- Les étapes sont représentées par des *nœuds* et les transitions par des *liens orientés* entre ces nœuds ;
- Les étapes de *test* sont représentées par des *losanges* ;
- Les étapes de *début* et de *fin* sont représentées par des *rectangles aux coins arrondis* ou par des *éclipses* ;
- Les étapes de *traitement* sont représentées par des *rectangles* ;
- Les appels à des *fonctions* ou *procédures* (aussi appelées *sous-routines*) sont représentés par des *rectangles dont les côtés sont dédoublés* ;
- Les étapes d'*entrée/sortie* sont représentées par des *parallélogrammes*.

L'inconvénient de cette représentation est qu'il est difficile de décrire des algorithmes complexes tout en gardant un diagramme lisible.

▪ **Le Pseudo Code** : Comme son nom l'indique, le pseudo code est une façon de décrire un algorithme en utilisant du texte ressemblant fortement à un langage de programmation, mais sans en être un. L'intérêt est de rester informel et indépendant d'un langage de programmation en particulier. Cette représentation permet aussi de ne pas représenter explicitement les détails triviaux de l'algorithme pour se concentrer sur l'essentiel.

Exemple : le même algorithme représenté sous forme d'organigramme puis de pseudo code :



```
Début
  Afficher "début du traitement "
  Compteur <- 0
  Tant que compteur < 4 Faire
    Traitement X
    Incréments le compteur
  Fin tant que
  Afficher "fin du traitement"
Fin
```

Algorithme : séquence finie d'actions permettant de résoudre un problème donné.

La notion d'algorithmique est directement dérivée du concept d'algorithme :

Algorithmique : ensemble des méthodes permettant de définir et/ou d'étudier des algorithmes.

10. Complexité d'un Algorithme

Il est souvent possible d'implémenter plusieurs solutions à un problème donné. La question se pose alors de savoir quel algorithme est le plus efficace. Et pour effectuer cette comparaison, il est nécessaire de déterminer sur quels critères elle doit être effectuée. Ainsi, il est indispensable d'introduire le champ de la *théorie de la complexité*, dont l'un des buts est de répondre à ce type de question.

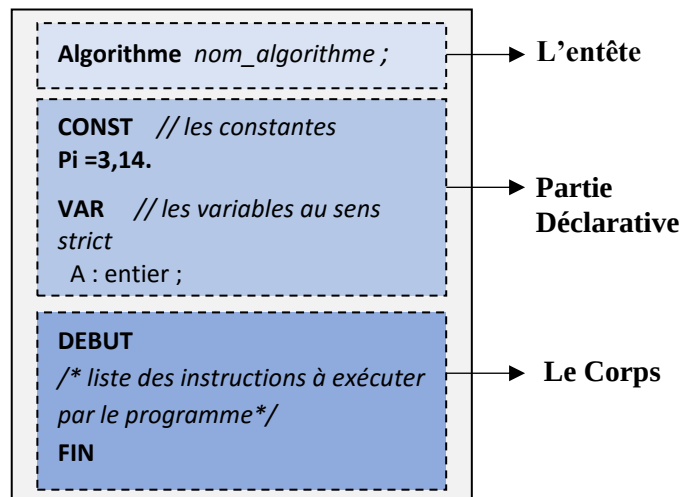
- ✓ **Théorie de la Complexité** : théorie développée dans le but de comparer efficacement différents algorithmes entre eux, Afin de savoir lequel était le meilleur pour un problème donné. Jusqu'aux années 70, on caractérisait les algorithmes de manière empirique, en considérant :
 - **La complexité temporelle**, (complexité en temps : Combien d'opérations va-t-on effectuer ?) équivaut au temps nécessaire pour l'exécution de l'algorithme.
 - **La complexité spatiale**, (complexité en espace : Quelle quantité de place en mémoire va-t-on utiliser ?) dite aussi encombrement mémoire, représente la place (espace mémoire) occupée (utilisée) par toutes les structures de données manipulées par l'algorithme.
 - **Les conditions d'exécution** :
 - Taille des données ;
 - Type d'ordinateur ;
 - Système d'exploitation ;
 - Langage de programmation.

Le problème de cette démarche est qu'elle ne permet pas de comparer les algorithmes de *manière objective*, car ils ne sont pas les seuls facteurs intervenant dans la performance de résolution du problème : d'autres facteurs extérieurs interviennent également (matériel, logiciel...). La solution retenue est de se détacher de ses facteurs extérieurs, et donc de l'implémentation de l'algorithme. Pour cela, une approche plus théorique doit être utilisée, reposant sur deux notions : les *opérations élémentaires* et les *positions mémoire*.

11. Structure Générale d'un Algorithme

Un algorithme est composé en 3 parties :

- **L'Entête** : constitué du mot algorithme, suivi d'un nom identifiant l'algorithme.
- **La Partie Déclarative** : comprend une liste de variables et des constantes (données) utilisées et manipulées dans le corps de l'algorithme.
- **Le Corps** : Comprend une suite d'instructions ou des tâches à exécuter.
- Les mots Algorithme, **DEBUT** et **FIN** ainsi que le **nom de l'algorithme** sont obligatoires.



12. Les Données d'un Algorithme

Un algorithme ou un programme manipule des informations qu'on appelle les données. On distingue deux types de données : les *constantes* et les *variables*. Comme leur nom l'indique, les "constantes" ont une valeur constante tout au long de l'exécution du programme. A l'inverse, les "variables" sont des données qui pourront varier (dont la valeur pourra changer) durant l'exécution du programme.

Les constantes et les variables sont des éléments fondamentaux et indispensables au bon déroulement d'un algorithme, caractérisés par un *identificateur* (*nom de la variable*), une *valeur* et un *type*.

12.1. Les Variables

Une *Variable* est une donnée (emplacement) stockée dans la mémoire de l'ordinateur. Elle est repérée par un identificateur (nom de la variable constitué de lettres et/ou de chiffres, sans espace) et contient une valeur dont le type (nature de la variable) peut être un entier, un réel, un booléen, un caractère, une chaîne de caractères... etc. Elle manipulée par un programme et pouvant être modifiée. Ce peut être :

- Une donnée d'entrée ;
- Le résultat final d'un calcul ;
- Un résultat intermédiaire de calcul.

Syntaxe de déclaration

```
VAR nom_de_variable : type ;
```

12.2. Les Constantes

Une *Constante*, comme une variable, peut représenter un chiffre, un nombre, un caractère, une chaîne de caractères, un booléen. Toutefois, contrairement à une variable dont la valeur peut être modifiée au cours de l'exécution de l'algorithme, la valeur d'une constante est *fixe* ne varie pas *tout le long de l'exécution d'un algorithme ou d'un programme*. Une constante est caractérisée par son nom et sa valeur (fixe).

Syntaxe de déclaration

```
CONST nom_de_constant = valeur ;
```

Exemple : CONST Pi = 3.141559 ;
Mois= "janvier" ;

13. Type de Données

Les variables que l'on utilise dans le programme ne sont pas toutes de même nature, il y a des nombres, des caractères, ... on dit que les variables sont typées.

Il est nécessaire de donner un type aux variables, car cela permet d'une part de contrôler leurs utilisations (on ne peut pas diviser un caractère par un nombre) et d'autre part indique combien de place faut-il réserver pour la variable (*taille*). Généralement les variables peuvent être des types suivants :

Ainsi, Le type d'une variable permet de déterminer le domaine des valeurs possibles que peut prendre cette variable. La connaissance du type permet également de déduire l'ensemble des opérateurs applicables sur les variables de ce type ainsi que l'espace mémoire en octets qui sera réservé à ces variables.

- **Booléen** : valeur pouvant être soit *vraie*, soit *fausse* codée sur un octet.
- **Entiers** : Il s'agit des variables destinées à contenir un nombre entier positif ou négatif. Généralement un entier occupe *deux octets*, ce qui limite les valeurs de -32768 à +32768. Cependant cela dépend des machines, des compilateurs et des langages. Certains langages distinguent les *entiers courts* (shortint :1octet), les *entiers longs* (Longint :4 octets) et les entiers simples (2 octets).
- **Réels** : Il s'agit des variables numériques qui ne sont pas des entiers, c'est à dire qui comportent des décimales. Généralement un nombre réel est codé généralement sur 4 octets. Il existe trois types de réel : *simple*, *double* et *Extended*.
- **Caractère** : octet correspondant à un code ASCII (alphanumérique). Les variables de type caractère contiennent des caractères alphabétiques ou numériques (de 0 à 9), mais dans ce cas ils ne sont pas considérés comme étant des nombres et on ne peut pas faire d'opérations dessus. Un caractère occupe 1 octet. *Convention* : les caractères sont entourés d'apostrophes (simples quotes) *Exemples* : 'a' 'B'.

- **Chaîne de caractères** : ensemble de caractères. *Convention* : les chaînes de caractères sont entourées de guillemets (doubles quotes). *Exemple* : " **les vacances**".
- **Tableau de données** : ensemble de données de même type (exemple : tableau d'entiers, tableau de réels).

Exemple :

Langage Algorithmique	Langage Pascal	Langage C
A, B : entier ; Note : réel ; Nom : chaîne de caractère ;	A, B : integer ; Note : real ; Nom : String[taille] ;	Int A, B ; Float Note ; Char [taille] ;

Remarque :

- "32" est différent de 32 car 32 est la valeur trente-deux par contre "32" représente la suite des chiffres 3 et 2 (chaîne de caractère).

Exemple de taille des types définis en Langage Pascal

Pascal définit 5 types d'entier :

Type	Intervalle	Occupation en mémoire
SHORTINT	de -128 à +127	1 octet
BYTE	de 0 à 255	1 octet
INTEGER	de -32768 à +32767	2 octets
WORD	de 0 à 65535	2 octets
LONGINT	de -2147483648 à 2147483647	4 octets

Pascal définit 4 types de réels :

Type	Valeurs autorisées	Occupation en mémoire
SINGLE	$1.5 * 10^{-45} .. 3.4 * 10^{38}$	4 octets
REAL	$2.9 * 10^{-39} .. 1.7 * 10^{38}$	6 octets
DOUBLE	$5.0 * 10^{-324} .. 1.7 * 10^{308}$	8 octets
EXTENDED	$3.4 * 10^{-4932} .. 1.1 * 10^{4932}$	10 octets

13.1. Convention De Nommage des Identifiants

Le nom d'un algorithme, d'une variable ou d'une constante doit respecter les règles suivantes :

- ✓ Commencer toujours par une lettre ;
- ✓ Ne comporter ni caractère spécial (comme l'espace) ni ponctuation ;
- ✓ Ne pas être un mot du langage algorithmique (comme « algorithme », « début », « fin », « variable », « non », « ou », « et », « si », « sinon », « pour »...).

13.1.1. Liste de Quelques Mots Réservés

And	Et	Else	Sinon	In	Dans	Then	Alors
End	Fin			Type	Type	To	Jusqu'à
Array	Tableau	Var	Var	With	Avec	Xor	Non ou
Begin	Début	Label	Etiquette	Procedure	Procédure	Program	Algorithme
Case	Cas	File	Fichier	Or	Ou	Try	Essai
Const	Const	Mod	Mod	Repeat	Répéter		
For	Pour	Nil	Nil	Set	Ensemble		
Record	Enregistrement	Function	Fonction	While	Tant que		
Div	Div	Goto	Aller a	Not	Non		
Do	Faire	If	Si	String	Chaîne		
Downto	Jusqu'à	Object	Objet	Of	De		

14. Les Instructions Élémentaires (de Base)

14.1. Lecture (Fonction d'Entrée)

La lecture est une instruction qui permet d'entrer des données tapées au clavier. Elle correspond à l'opération qui permet de saisir des valeurs pour qu'elles soient utilisées par le programme. Cette instruction est notée « *saisir identificateur* » ou « *lire identificateur* ».

Syntaxe

```
Lire (variable1) ;  
Lire (variable1, variable2) ;
```

Exemple :

- ✓ **Lire** (note) ; **Lire** (A, B) ; En langage Algorithmique.
- ✓ **Read** (note) ; **Read** (A, B) ; En langage Pascal.
- ✓ **Scanf**(¬e) ; **Scanf**(&A, &B) ; En langage C.

14.2. L'écriture (Fonction de Sortie)

Instruction qui permet d'afficher le contenu d'une variables ou/et un message sur l'écran. Ainsi, elle permet l'affichage des valeurs des variables après traitement. Cette instruction est notée « *afficher identificateur* ».

Syntaxe

```
Ecrire (variable) ; ou Afficher (variable)  
Ecrire (" message") ;  
Ecrire ("message", variable) ;
```

Exemple :

- ✓ **Ecrire** (A) ; **Ecrire** ("donnez votre nom : ") ; En langage Algorithmique.
- ✓ **Write** (A) ; **Write** ('donnez votre nom : ') ; En langage Pascal.
- ✓ **Printf**(A) ; **Printf** ("donnez votre nom : ") ; En langage C.

14.3. L'Affectation

L'affectation est une **opération** qui consiste à attribuer une valeur à une variable. Elle est représentée par une flèche orientée à gauche « **←** » ou par le symbole « **:=** » en langage Pascal ou « **=** » en langage C.

Syntaxe

```
Variable := valeur ou expression ;  
Variable ← valeur ou expression ;
```

Exemple

- ✓ **A ← 2** : la variable A reçoit la valeur 2. (en Algorithmique)
- ✓ **B := A+1** : la variable B reçoit le contenu de A plus 1. (En langage Pascal)
- ✓ **Nom = "Mohamed"** : la variable Nom reçoit la valeur Mohamed. (En langage C)

Remarque :

- L'instruction d'affectation ne modifie que ce qui est situé à gauche de la flèche.
- La valeur de la partie droite doit obligatoirement être du type de la variable dont la valeur est modifiée.

15. Expressions et Opérateurs

15.1. Expression

Une **expression** est un ensemble de valeurs reliées par des opérateurs, et équivalent à une seule valeur
⇒ Une expression simple est constituée d'opérandes reliés par des opérateurs. Nous distinguons deux types d'expressions :

- ✓ Les **expressions arithmétiques** donnant une valeur numérique. Exemples : $4+2$, $5*6$
- ✓ Les **expressions logiques** donnant une valeur booléenne. Exemples : $4<9$, $A>6$, A ou B

15.2. Les Opérateurs

Les opérateurs sont des signes qui relient deux valeurs, pour produire un résultat. Les opérateurs permettent d'élaborer une expression en vue d'effectuer un calcul ou une comparaison.

Exemple :

- ✓ Opérateurs arithmétiques : $+$, $-$, $*$, $/$, $\wedge$, $<$, ...etc.
- ✓ Opérateurs logiques : et, non, ou, ...etc.
- ✓ Dans « $4+2$ », « $+$ » désigne l'opérateur ; « 4 » et « 2 » sont les opérandes.

Il existe plusieurs types d'opérateurs :

Nature	Variables utilisées	Notation	Signification
Opérateurs arithmétiques	Entier Réel	+	Addition
		-	Soustraction
		*	Multiplication
		/	Division (réelle)
		DIV	Division entière
		MOD	Reste de la division entière
Opérateurs logiques	Booléen Entier	et	Fonction ET
		ou	Fonction OU
		ouex	Fonction OU EXCLUSIF
		non	Fonction NON
Opérateur de concaténation	Chaîne de caractères	+	Concaténation
Opérateurs de comparaison Ou relationnels	Booléen Entier Réel Caractère Chaîne de caractères	=	Egal
		<>	Différent
		<	Inférieur
		>	Supérieur
		≤	Inférieur ou égal
		≥	Supérieur ou égal

15.3. Évaluation d'une Expression (Priorité Des Opérateurs)

Lors de l'évaluation d'une expression, on tient compte de la priorité entre les opérateurs. A chaque opérateur est associée une priorité. Lors de l'évaluation d'une expression, la priorité de chaque opérateur permet de définir l'ordre d'exécution des différentes opérations. Aussi, pour lever toute ambiguïté ou pour modifier l'ordre d'exécution, on peut utiliser des parenthèses.

- **Ordre de priorité décroissante des opérateurs arithmétiques et de concaténation :**
 - « $\wedge$ » (élévation à la puissance)
 - « $-$ » (changement de signe)
 - « $\times$ », « $/$ » et «DIV»
 - «MOD»
 - « $+$ » et « $-$ »
 - « $+$ » (concaténation)
- **Ordre de priorité décroissante des opérateurs logiques :**
 - ✓ «non»
 - ✓ «et»
 - ✓ «ou»
 - ✓ «ou_exclusif».

D'une manière générale on aura :

- Niveau 1 : NOT.
- Niveau 2 : *, /, mod, div, AND.
- Niveau 3 : +, -, OR, XOR.
- Niveau 4 : =, <, >, <=, >=, <>.

Remarques :

- ✓ Les opérations entre parenthèses sont prioritaires.
- ✓ L'usage des parenthèses est vivement conseillé dans le cas d'expressions complexes.
- ✓ Dans le cas où deux opérateurs ont la même priorité, on commence par la plus à gauche.

Retenons

Une expression est composée d'opérateurs et d'opérandes et son évaluation produit une valeur.

On distingue les expressions arithmétiques, et les expressions logiques.

L'évaluation d'une expression se fait toujours selon l'ordre de priorité des opérateurs.

Table de vérité des opérateurs booléens

<i>A</i>	<i>B</i>	<i>A ET B</i>	<i>A OU B</i>	<i>NON A</i>
Faux	Faux	Faux	Faux	Vrai
Faux	Vrai	Faux	Vrai	Vrai
Vrai	Faux	Faux	Vrai	Faux
Vrai	Vrai	Vrai	Vrai	Faux

15.4. Les Opérations

Syntaxe	Utilisation	Type des variables	Description
Inc(a);	Procédure	Tout scalaire	Le nombre a est incrémenté de 1
Inc(a,n);	Procédure	Tout scalaire	Le nombre a est incrémenté de n
Dec(a);	Procédure	Tout scalaire	Le nombre a est décrémenté de 1
Dec(a,n);	Procédure	Tout scalaire	Le nombre a est décrémenté de n
Trunc(a);	Fonction	Tout réel	Prise de la partie entière du nombre a sans arrondis
Int(a);	Fonction	a : tout réel Int(a) : Longint	Prise de la partie entière du nombre a sans arrondis
Frac(a);	Fonction	Tout réel	Prise de la partie fractionnaire du nombre a
Round(a);	Fonction	a : tout réel Round(a) : Longint	Prise de la partie entière du nombre a avec arrondi à l'unité la plus proche
Pred(x);	Fonction	Tout scalaire	Renvoie le prédécesseur de la variable x dans un ensemble ordonné
Succ(x);	Fonction	Tout scalaire	Renvoie le successeur de la variable x dans un ensemble ordonné
High(x);	Fonction	Tout scalaire	Renvoie la plus grande valeur possible que peut prendre de la variable x
Low(x);	Fonction	Tout scalaire	Renvoie la plus petite valeur possible que peut prendre de la variable x
Odd(a);	Fonction	a : entier Odd(a) : Boolean	Renvoie true si le nombre a est impair et false si a est pair
SizeOf(x);	Fonction	x : tous SizeOf(x) : Integer	Renvoie le nombre d'octets occupés par la variable x

16. Concevoir Un Algorithme

Pour concevoir un algorithme, il faut suivre les étapes suivantes :

Etape 1

- ✓ Lire et comprendre bien l'énoncé du problème à résoudre

Etape 2

- ✓ Définir les résultats du problème (les sorties) ;
- ✓ Définir les données du problème (les entrées) ;
- ✓ Définir le traitement (les relations permettant d'obtenir les résultats à partir des données) ;

Etape 3

- ✓ Ecrire l'algorithme en respectant la structure pseudo code

Exemple : Ecrire un algorithme qui demande la valeur du rayon pour calculer la surface d'un cercle

Solution

- **Etape 1 :** On veut calculer la surface d'un cercle
- **Etape 2 :**
 - ✓ **Résultat :** La surface du cercle Surf
 - ✓ **Données :** Le rayon R, 3.14 Pi
 - ✓ **Traitement :** $\text{Surf} = \text{Pi} * \text{R}^2$.
- **Etape 3 :** La résolution du problème en: (Algorithme, Langage Pascal et Langage C).

```
Algorithme Surface_cercle ;      L'entête
CONST Pi = 3,14 ;
VAR R, Surf : Réel ;             } Les Déclarations
DEBUT
  Ecrire ("Donnez la valeur du rayon : ") ;
  Lire (R) ;
  Surf ← Pi*R^2 ;
  Ecrire (" La surface de cercle est : ", Surf) ;
FIN.                             } Le Corps
```

```
Program Surface_cercle ;
CONST Pi = 3.14 ;
VAR R, Surf : Real ;
BEGIN
  Write (' Donnez la valeur du rayon : ');
  Read (R) ;
  Surf := Pi*R^2 ;
  Write (' La surface de cercle est : ', Surf) ;
END.
```

Surface_cercle.c

```
#include<stdio.h>
#include<math.h>
Main()
{
  Const float Pi = 3.14 ;
  Float R, Surf ;

  Printf (" Donnez la valeur du rayon : \n") ;
  Scanf ("%f ", &R) ;
  Surf = Pi*R^2 ;
  Printf (" La surface de cercle est : %f ", Surf) ;
}
```